# MAS Final Project: Environment Dynamics (`env.py`) Complete Code Documentation

## 1. Scope

This chapter documents `env.py` end-to-end, including:

- imports and module initialization,
- `WarehouseEnv` constructor and state layout,
- reset/spawn logic,
- event handling,
- state encoding,
- step transition and reward logic,
- conflict detection and telemetry,
- rendering pipeline,
- evaluation API,
- helper methods and data contracts.

## 2. Module Imports and Globals

Imports:

- `math`
- `random`
- `typing` (`Dict`, `List`, `Optional`, `Tuple`)
- `pygame`
- `pygame.gfxdraw`
- `config as cfg`
- `Action`, `Robot` from `agent.py`

Module-level side effects:

- `pygame.init()` is called at import time.

Type alias:

- `GridPos = Tuple[int, int]`

## 3. Class Overview: `WarehouseEnv`

`WarehouseEnv` is the runtime world model.
Responsibilities:

- maintain robot/shelf/grid state,
- execute simultaneous action step semantics,
- produce per-agent state vectors and rewards,
- detect and record conflicts/collisions,
- provide rendering and evaluation utilities.

## 4. Constructor (`__init__`)

Signature:

- `__init__(self, render: bool = True, num_agents: Optional[int] = None)`

Fields initialized from config:

- `grid_w`, `grid_h`
- `GOALS`
- `num_shelves`
- `num_agents` (override allowed by argument)
- `render_enabled`
- `render_fps`

If rendering enabled:

1. Chooses cell size relative to target 2560x1440 and config limit.
2. Computes pixel dimensions:
   - `grid_width_px = grid_w * cell_size`
   - `grid_height_px = grid_h * cell_size`
3. Creates pygame window and clock.
4. Initializes scaled fonts:
   - `font`
   - `font_small`
   - `font_tiny`
5. Stores `cell_size`.

Final constructor action:

- calls `self.reset()`.

## 5. Random Position Utility

Method:

- `get_random_free_position(self, occupied_positions: set) -> Tuple[int, int]`

Behavior:

- samples random `(x, y)` uniformly inside grid until position not in `occupied_positions`.

Used by reset and shelf respawn on delivery.

## 6. Episode Reset (`reset`)

Signature:

- `reset(self) -> List[List[float]]`

Flow:

1. Initialize `occupied` with goal cells.
2. Spawn `self.num_agents` robots on free cells.
3. Spawn `self.num_shelves` shelves on free cells with dict fields:
   - `id`
   - `x`
   - `y`
   - `carried=False`
   - `requested=True`
4. Reset counters and telemetry:
   - `steps`
   - `last_collisions`
   - `last_delivered`
   - `last_conflicts`
   - `selected_agent_id`
   - `_planner_allowed_shelf_entries`
   - `_planner_last_actions`
   - `_planner_debug_by_agent`
   - `event_log`
   - `traffic_heat`
   - `collision_heat`
   - `total_collisions`
   - `total_deliveries`
5. Returns list of per-robot state vectors via `get_state`.

## 7. UI Event Handling

Method:

- `handle_event(self, event) -> None`

If rendering disabled:

- returns immediately.

Supported interactions:

1. Mouse left-click:
   - if click inside grid,
   - select robot on clicked cell (if present),
   - updates `selected_agent_id`.
2. Keyboard:
   - `TAB`, `RIGHT`, `.` -> cycle selected agent forward
   - `LEFT`, `,` -> cycle selected agent backward

Helper:

- `_cycle_selected_agent(delta)`

## 8. Agent Selection Helper

Method:

- `_cycle_selected_agent(self, delta: int) -> None`

Behavior:

1. If no robots: sets selected id to `-1`.
2. Builds sorted robot id list.
3. If current selected id missing: selects first id.
4. Else rotates index by `delta` modulo robot count.

## 9. Observation Encoding (`get_state`)

Method:

- `get_state(self, robot: Robot) -> List[float]`

Output composition:

1. Robot self features (4):
   - normalized `x`
   - normalized `y`
   - normalized direction (`dir.value / 3.0`)
   - carrying flag
2. Shelf block (`num_shelves * 4`):
   - shelves sorted by shelf id,
   - each contributes normalized x/y + carried/requested flags,
   - padded with zeros if fewer shelves than configured.
3. Other robots block (`(num_agents - 1) * 4`):
   - each contributes normalized x/y + dir + carrying flag,
   - padded with zeros if needed.

Normalization uses `_normalize(value, max_value)`.

## 10. Distance-to-Target Helper (`get_dist_to_target`)

Method:

- `get_dist_to_target(self, robot: Robot) -> Optional[float]`

Cases:

1. Robot carrying requested shelf:
   - returns min Manhattan distance to any goal.
2. Robot carrying non-requested shelf:
   - returns `None`.
3. Robot empty:
   - returns min Manhattan distance to requested uncarried shelf.
4. No applicable targets:

- returns `None`.

Used in reward shaping after action execution.

## 11. Core Transition Method (`step`)

Signature:

- `step(self, actions: List[int], record_trajectories: bool = False)`

Returns:

- `states`
- `rewards`
- `done`
- `collisions`
- `trajectories` (or `None`)

### 11.1 Initialization and Decode

At step start:

1. Decode each integer action into `Action` via `_decode_action`.
2. Store decoded values in `_planner_last_actions`.
3. Reset per-step event container.
4. Clear `last_conflicts`.
5. Populate `last_conflicts` from intent analysis:
   - `_record_intent_conflicts(...)`.
6. Optionally initialize trajectory buffers.
7. Cache old target distances for shaping.
8. Initialize base reward `-0.01` per robot.

### 11.2 First Pass: Non-Forward Actions + Forward Intent Collection

For each robot:

- `FORWARD`:
  - compute intended cell,
  - if boundary violation -> early blocked, collision penalty path,
  - if shelf-blocked -> early blocked,
  - else store forward intent.
- `TURN_LEFT`:
  - rotate left and small penalty `-0.002`.
- `TURN_RIGHT`:
  - rotate right and small penalty `-0.002`.
- `PICK_DROP`:
  - delegate to `robot.pick_or_drop(self)`,

  - apply returned reward,
  - log events, increment `delivered_count` if delivered.
- `WAIT`:
  - small penalty `-0.003`.

Early blocked forward moves:

- get additional `-0.2`,
- increment `collisions`,
- increment `collision_heat` at current cell.

## 11.3 Forward Conflict Resolution

Build static occupied positions from robots not moving forward.

Mark forward moves blocked if:

1. target cell occupied by static robot.
2. multiple movers target same cell (vertex contention).
3. two movers perform edge swap:
   - `from_a == to_b` and `to_a == from_b`.

For blocked forward moves:

- reward `-0.2`,
- increment `collisions`,
- increment `collision_heat`,
- append event `blocked by robot conflict`.

For unblocked forward moves:

- update robot position,
- if carrying shelf, move shelf with robot,
- add small positive move reward `+0.01`.

## 11.4 Heatmaps and Shaping

After moves:

1. Increment `traffic_heat` for each robot's final cell.
2. For each robot, compare old/new target distance:
   - if improved: bonus `+0.12 * improvement`
   - if worsened: smaller penalty `+0.04 * negative_improvement` (net subtraction)
3. If carrying non-requested shelf:
   - extra penalty `-0.05`.

Team delivery bonus:

- if `delivered_count > 0`, add `2.0 * delivered_count` to every robot reward.

### 11.5 Finalization

Update counters:

- `steps += 1`
- `last_collisions`
- `last_delivered`
- `total_collisions += collisions`
- `total_deliveries += delivered_count`

Done condition:

- `done = self.steps > 1000`

Event log:

- append up to last 6 step events prefixed with timestep,
- cap `event_log` length to 28 entries.

Build `states` via `get_state` for all robots.
If trajectories enabled, append final robot positions.

Return tuple `(states, rewards, done, collisions, trajectories)`.

## 12. Conflict Tracking Helper (`_record_intent_conflicts`)

Purpose:

- produce structured conflict records from action intents before final motion resolution.

Process:

1. Build forward intents (`agent_id -> (from, to)`), reporting `boundary` conflicts early.
2. Group intents by target cell:
   - if multiple agents target same cell -> `vertex` conflict.
3. Pairwise check for swaps:
   - if edges opposite -> `edge` conflict.

Conflict schema examples:

- `{"type":"boundary","agent":id,"from":(x,y),"to":(nx,ny)}`
- `{"type":"vertex","agents":[...],"pos":(x,y)}`
- `{"type":"edge","agents":[a,b],"from":(x1,y1),"to":(x2,y2)}`

These records are consumed by verification/refinement modules.

## 13. Rendering Pipeline (`render`)

Method:

- `render(self) -> None`

If rendering disabled:

- returns immediately.

Draw sequence:

1. Clear background with `cfg.BG_PURE`.
2. Draw subtle grid lines.
3. Draw goals with shadow, border, and label.
4. Draw shelves (non-carried, non-under-robot):
   - requested shelves highlighted differently.
5. Detect robot orbits via `detect_role_orbits` (best-effort try/except fallback).
6. Draw robots:
   - anti-aliased body,
   - direction arrow,
   - carrying indicator,
   - orbit corner marker color.
7. Draw conflict overlays from `last_conflicts`:
   - red transparent cell for vertex conflict,
   - red line for edge conflict.
8. `pygame.display.flip()`.
9. frame cap by `render_fps` via `clock.tick`.

Rendering does not mutate planning state beyond visual presentation.

## 14. Evaluation API (`evaluate`)

Method:

- `evaluate(self, planner, num_episodes=50, max_steps_per_episode=200, progress_every=0, logger=None) -> float`

Flow:

1. Loop episodes:
   - `reset`,
   - step planner until done or step cap,
   - accumulate collisions.
2. Optional progress logging/printing.
3. Return:
   - `total_collisions / (num_episodes * num_agents)`
   - with guards returning `0.0` when episodes or agents are non-positive.

Used by `main.py` eval mode.

## 15. Helper Methods

### 15.1 `_normalize(value, max_value)`

- denominator is `max(max_value - 1, 1)`
- returns value/denominator

Keeps features in approximately `[0,1]`.

### 15.2 `_decode_action(action_idx)`

- attempts `Action(action_idx)`,
- falls back to `Action.WAIT` on exception.

Protects env from invalid action values.

## 16. Reward Model Summary

Per-step reward components in `step`:

1. base living penalty: `-0.01`
2. wait penalty: `-0.003`
3. turn penalty: `-0.002`
4. move success bonus: `+0.01`
5. blocked/collision penalty: `-0.2`
6. distance shaping:
   - stronger positive weight when getting closer (`0.12`)
   - weaker negative when moving away (`0.04`)
7. carrying wrong shelf penalty: `-0.05`
8. pick/deliver/drop event reward from agent:
   - pick requested shelf: `+5.0`
   - deliver requested shelf: `+20.0`
   - drop: `-0.1`
   - noop pick/drop attempts: `-0.05`
9. team delivery bonus:
   - `+2.0 * delivered_count` added to all agents.

## 17. Planner-Environment Contracts

Environment fields used by planner:

- `robots`, `shelves`, `GOALS`, `grid_w`, `grid_h`

Environment fields expected from planner:

- `_planner_allowed_shelf_entries`
- `_planner_last_actions`
- `_planner_debug_by_agent`

Important integration point:

- `Robot._occupied_by_shelf` reads `_planner_allowed_shelf_entries` to allow only planner-approved shelf entry.

## 18. Verification/Refinement Integration Points

`verification.py` reads:

- collision counts from `step` return,
- structured conflict records from `env.last_conflicts`.

`refinement.py` consumes these conflict records to inject planner constraints. Therefore, conflict record semantics in env are safety-critical interfaces.

## 19. Determinism and Randomness

Deterministic within episode given fixed initial state and actions.

Randomized components:

- initial robot spawn,
- initial shelf spawn,
- robot initial directions (via `Robot` constructor).

Global reproducibility depends on seed setting from `main.py`.

## 20. Complexity Notes

Per-step heavy parts:

1. pairwise forward conflict checks (worst-case quadratic in moving robots),
2. state vector generation scales with configured shelves and agents,
3. rendering cost dominates when enabled.

Memory-heavy telemetry:

- heatmaps size `grid_h * grid_w`,
- bounded `event_log`.

## 21. Full Method Coverage Checklist

All methods documented:

- `__init__`
- `get_random_free_position`
- `reset`
- `handle_event`
- `_cycle_selected_agent`
- `get_state`
- `get_dist_to_target`
- `step`
- `render`

- `evaluate`
- `_normalize`
- `_decode_action`
- `_record_intent_conflicts`

## 22. Chapter Summary

`env.py` is the authoritative transition system of the project.
It defines how planned actions become actual state changes, rewards, conflicts, and diagnostics.
Because verification and refinement rely on `step` outputs and `last_conflicts`, the correctness of environment semantics is central to the entire safety loop.